

i have one is that when i normalize a text like senior c# developer with sql and azure that remove # and return c that is different meaning

Excellent observation. This is a very common issue in NLP, especially with programming-related text.

For example, suppose your normalization removes all punctuation:

```
Original:
Senior C# Developer with SQL and Azure
```

After normalization:

```
senior c developer with sql and azure
```

Now **C#** becomes **C** , which changes the meaning completely.

Similarly:

Original	After removing punctuation	Problem
C#	C	✗ Different language
C++	C	✗ Different language
ASP.NET	aspnet or asp net	✗ May lose meaning

.NET	net	✗ "net" is a common English word
Node.js	nodejs or node js	✗ Different token
React.js	reactjs	Less severe but inconsistent
Entity Framework Core	entity framework core	✓ Fine

The solution

For a **Resume Search Engine**, you should **not blindly remove punctuation**. Instead, preserve technology names.

Option 1 (Recommended): Protect keywords before normalization

Create a dictionary of technologies:

```
C#      -> csharp
C++     -> cpp
ASP.NET -> aspnet
.NET    -> dotnet
Node.js -> nodejs
React.js-> reactjs
```

Example:

Original

Senior C# Developer using ASP.NET Core and .NET 8

Replace first:

```
Senior csharp Developer using aspnet Core and dotnet 8
```

Then normalize:

```
senior csharp developer using aspnet core and dotnet 8
```

Nothing important is lost.

Option 2: Modify your regex

If you're currently doing something like:

```
Regex.Replace(text, @"[\w\s]", "");
```

that removes `#`, `+`, and `.`.

Instead, allow those characters:

```
Regex.Replace(text, @"[^a-zA-Z0-9\s#.+]", "");
```

Now:

```
C#  
C++  
ASP.NET  
.NET  
Node.js
```

remain intact.

Option 3 (Best for production)

Maintain a technology dictionary:

```
C#  
C++  
.NET  
ASP.NET  
ASP.NET Core  
SQL Server  
Azure  
Azure DevOps  
Entity Framework
```

```
Entity Framework Core
React.js
Node.js
Angular
Blazor
MAUI
Redis
MongoDB
PostgreSQL
Docker
Kubernetes
```

Replace them with normalized forms before tokenization:

```
C#           -> csharp
C++          -> cpp
ASP.NET Core -> aspnetcore
ASP.NET      -> aspnet
.NET         -> dotnet
SQL Server   -> sqlserver
Azure DevOps -> azuredevops
Entity Framework Core -> entityframeworkcore
```

Then continue with tokenization and TF-IDF.

Recommendation for your project

Since you're building this project in C#, I recommend adding a preprocessing step like:

```
Raw Resume
  ↓
Technology Replacement
  ↓
Normalize Text
  ↓
Tokenization
  ↓
Stop Word Removal
  ↓
Lemmatization
  ↓
TF-IDF
```



Cosine Similarity

This is a common approach in search engines because it preserves the meaning of technical terms while still cleaning the text for indexing and matching.